1. IDENTIFICACIÓN DEL ARTÍCULO · SCIENTIFIC WORKSHOP — GESTIÓN DE SISTEMAS DE DATOS MASIVOS (ETSISI - UPM)

Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes (Sistema EVAPORATE)

Autores del artículo: Simran Arora, Brandon Yang*, Sabri Eyuboglu*, Avanika Narayan, Andrew Hojel (*Stanford University*), Immanuel Trummer (*Cornell University*) y Christopher Ré (*Stanford University*)
Publicación: *Proceedings of the VLDB Endowment (PVLDB)*, Vol. 17, No. 2, pp. 92–105, 2023 (ISSN: 2150-8097) · **DOI:** https://doi.org/10.14778/3626292.3626294 · **PDF VLDB:** vldb.org/pvldb/vol17/p92-arora.pdf · **Código:** github.com/HazyResearch/evaporate

CONGRESO CORE A* | REVISTA JCR Q1 | BLOQUE I · ALMACENAMIENTO / ETL-ELT

Análisis realizado por (Grupo de Trabajo - UPM):
Integrantes: *[Añadir nombres de los 3–4 estudiantes del grupo]*
Máster Universitario en Aprendizaje Automático y Datos Masivos

2. Motivación y Problema de Investigación — Apartado 2

Las organizaciones almacenan información crítica en **Data Lakes heterogéneos** compuestos por millones de documentos no estructurados o semi-estructurados (páginas web HTML, informes médicos o regulatorios en PDF, correos en texto plano). En su estado crudo, **estos datos no permiten ejecutar consultas analíticas SQL/OLAP**.

> **Definición formal del problema:** Dada una colección de n documentos heterogéneos $D = \{d_1, d_2, \ldots, d_n\}$ sin esquema previo, construir automáticamente una **vista estructurada (tabla relacional)** definida por un esquema de atributos $A = \{a_1, \ldots, a_m\}$ y n registros extraídos $R = \{r_1, \ldots, r_n\}$ (donde cada r_i es una m-tupla), **sin entrenamiento específico de dominio ni etiquetado manual**.

El Trilema de los Sistemas de Extracción e Ingesta

1. Generalidad	2. Coste Escalable	3. Calidad Analítica
Soportar múltiples formatos (HTML, PDF, TXT) y dominios sin entrenar modelos ad-hoc.	Escalar a millones de documentos sin disparar el coste computacional ni monetario.	Atributos de alta cobertura y valores normalizados listos para consultas SQL.

¿Por qué fracasan los sistemas del estado del arte (SoTA) previos?

- **Sistemas Web/DOM (DOM-LM, Ceres):** Asumen que los atributos están siempre en nodos hoja del árbol `HTML-DOM` y exigen entrenamiento supervisado en cada dominio. **No funcionan en PDFs ni en texto plano (TXT)**.
- **Sistemas NLP / OpenIE clásicos (OpenIE6):** Dependen de etiquetadores gramaticales (POS, NER, árboles de dependencias) que se degradan drásticamente en documentos largos (ej. informes médicos de 20 páginas de la FDA) y extraen centenares de tripletas sin normalizar (427 relaciones distintas en un solo informe FDA).
- **Pipelines ETL manuales:** Requieren expertos humanos programando reglas frágiles para cada fuente.

6. Relación con los Contenidos de la Asignatura — Apartado 6

El artículo conecta transversalmente los **tres bloques** de *Gestión de Sistemas de Datos Masivos* y amplía directamente lo estudiado en clase:

- **Bloque I — Fuentes de Datos y Web Scraping (*B1_1_4_Fuentres De Datos.pdf*):**
  - Resuelve el problema de la diapositiva 6 (*"From unstructured data to actionable data"*) y el caso de estudio de la diapositiva 14 (**informes médicos/FDA** difíciles de estructurar).
  - Automatiza la programación de los parsers de **Web Scraping** vistos en las diapositivas 20–32: en vez de que el ingeniero escriba a mano código con `BeautifulSoup` (`soup.find_all`), `lxml` /XPath o expresiones regulares (`re.compile`), el LLM sintetiza automáticamente esas mismas funciones Python.
- **Bloque I — Arquitecturas ETL/ELT y Transformaciones (*B2_2_2_Gestión De Informacion.pdf*):**
  - Compara empíricamente una transformación **Data-driven pura** (*Evaporate-Direct*: procesar el contenido documento a documento con Deep Learning/LLM, diap. 27) frente a una transformación **Model-driven / basada en reglas** sintetizada automáticamente (*Evaporate-Code+*, diap. 26).
  - Resuelve la desventaja clave de los sistemas ETL/ELT citada en la diapositiva 14: *"una función rara vez cubre el 100% de los casos potenciales"* mediante el ensamblado estadístico de múltiples funciones con **Weak Supervision**.
- **Bloque II — Recuperación de Información y LLMs (*2_1_1* y *2_2_RAG.pdf*):**
  - Aplica **Chunking** para respetar la ventana de contexto del LLM, búsqueda léxica por palabras clave, vectorización **TF-IDF + K-Means** (Fig. 5) para agrupar documentos mezclados por fuente sin supervisión, y evalúa con **Precision, Recall y F1** analizando modos de fallo y alucinaciones del LLM.
- **Bloque III — Gobernanza e Interoperabilidad (*Block 3.1 / 3.2*):**
  - Aborda el descubrimiento automático de esquemas (*Schema Discovery*) y la tensión entre extracción abierta (*OpenIE*, tripletas sin ontología previa) y cerrada (*ClosedIE*).

3. Propuesta y Contribución Científica: EVAPORATE-CODE+ — Apartado 3

Los autores exploran el uso de *In-Context Learning* en **Large Language Models (LLMs)** mediante 6 plantillas de *prompts* agnósticos al dominio y descubren un nuevo espacio de compromiso (*trade-off*) entre **Coste y Calidad**:

Comparativa de las 3 Arquitecturas Propuestas

Estrategia	Mecanismo de Funcionamiento	Coste LLM (Tokens)	Calidad Media (Pair F1)
1. Evaporate-Direct	El LLM lee cada documento completo (en *chunks*) y extrae directamente pares `<atributo>: <valor>` .	**Lineal O(n) · ¡Carísimo! (1,1 M$ en Wikipedia)**	56.6 F1 (Inconsistente entre docs)
2. Evaporate-Code	Muestrea $k \ll n$ docs (k=*10*) y pide al LLM **1 función Python** por atributo para ejecutarla a escala en CPU.	**Sublineal O(m · k) (Independiente de n)**	**42.8 F1 (-13.8 pts) (Funciones frágiles)**
3. Evaporate-Code+	**Sintetiza múltiples funciones candidatas diversas por atributo y agrega sus salidas con Weak Supervision.**	**Sublineal O(m · k) (Reducción de 110×)**	**66.7 F1 (+10.1 pts vs Direct y +25.1 pts vs Code)**

Arquitectura en 3 Etapas de EVAPORATE-CODE+ (Flujo de Ejecución):

Etapa 1: Schema Synthesis
Toma una muestra pequeña $\tilde{D}$ de $k = 10$ documentos. El LLM identifica atributos candidatos explícitos, calcula su frecuencia y los re-rankea priorizando atributos de alta cobertura (*head attributes*).

Etapa 2: Diverse Code Synthesis
Para cada atributo a_i, usa múltiples plantillas de prompt (P_A orientada a `regex` sin ejemplos y P_B con ejemplos que importan librerías como `BeautifulSoup`) sobre distintos pasajes para generar un conjunto diverso de funciones $F = \{f_1, \ldots, f_k\}$.

```
from bs4 import
BeautifulSoup
def get_draft_year(doc):
  soup =
BeautifulSoup(doc,
"html.parser")
  return
soup.find(...).text
```

Etapa 3: Function Aggregation (Alg. 1)
Ejecuta las funciones en CPU (0,00025 s/doc) y combina sus "votos" sin datos etiquetados usando el **Algoritmo 1 de Supervisión Débil (Weak Supervision)** para filtrar código erróneo y resolver conflictos.

Innovación Algorítmica: ¿Cómo adapta el Algoritmo 1 la Supervisión Débil (WS) a código generado por IA?

La teoría clásica de *Weak Supervision* (Snorkel) asume funciones diseñadas por humanos con precisión >50%, abstención explícita y clases discretas. En código generado por LLMs, **el 51% de las funciones tiene <50 F1 (y el 40% tiene <25 F1)**. El **Algoritmo 1** lo resuelve en 4 pasos sobre una muestra de validación *D_eval* (≤10 docs) comparando cada función f_j con la extracción directa del LLM F:

- **1. Manejo de Abstenciones (tasa e):** Calcula la fracción e de documentos en *D_eval* donde el LLM extrae un valor no vacío. Si $e > \tau$ (el atributo casi siempre existe), una salida vacía de f_j se interpreta como *abstención* (la función no supo extraerlo); si $e \leq \tau$, se interpreta como predicción legítima de "valor ausente".
- **2. Puntuación y Filtrado de Funciones Ruidosas:** Evalúa la calidad empírica $\hat{a}_j$ de cada función frente al LLM en *D_eval* y **descarta todas las funciones con $\hat{a}_j \leq 0.5$**, garantizando el supuesto teórico de WS (Proposición 1 del paper).
- **3. Discretización del Espacio Abierto de Texto:** Como las funciones devuelven texto libre arbitrario, agrupa los valores únicos extraídos para cada documento en los *top-b buckets* más frecuentes y marca el resto como abstenciones.
- **4. Modelo Gráfico Latente:** Aprende sin etiquetas las precisiones y correlaciones entre las funciones filtradas F' para emitir el valor final consensuado $\hat{y}_i$ de cada celda de la tabla.

> **Puntos de corte asintóticos (Crossover Points - Figura 3):** El coste de *Evaporate-Code+* crece con el nº de atributos m, no con el nº de documentos n. **Supera en eficiencia a *Evaporate-Direct* a partir de solo $n > 40$ documentos** (y siempre que el esquema tenga menos de $m < 2.500$ atributos).

4. Evaluación Experimental y Resultados — Apartado 4

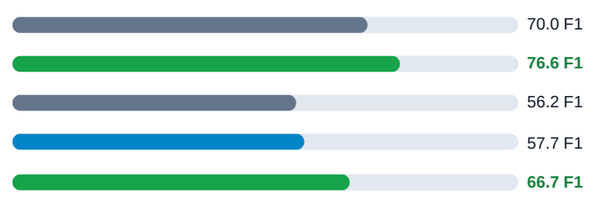
Configuración experimental: Evaluado en **16 benchmarks reales** de **5 dominios** y **3 formatos**: *SWDE Movies* (8 webs HTML, 1.063–2.000 págs/web), *SWDE Universities* (5 webs HTML), *Wikipedia NBA Players* (HTML complejo con infoboxes), *Enron Emails* (500.000 correos TXT) y *FDA 510(k) Medical Device Reviews* (PDFs convertidos a TXT de ~20 páginas). Evaluado con 4 LLMs (`text-davinci-003` , `GPT-4` , `Claude-v1` , `Jurassic-2`) usando métricas **Pair F1** (OpenIE end-to-end) y **Text F1** (ClosedIE).

110×	+10.1 F1	+25.1 F1	0.25 ms
Reducción de Tokens (y 378× en tamaño real)	Mejora (+18%) sobre Evaporate-Direct	Mejora del Algoritmo 1 sobre Evaporate-Code	Tiempo de ejecución por función/doc en CPU

Resultado 1: Calidad (Pair F1) y Coste por cada 10.000 Documentos (Tabla 3)

Dataset (Formato)	Direct F1	Direct Coste ($)	Code+ F1	Code+ Coste ($)	Mejora F1	Ahorro Tokens
FDA 510(k) (PDF/TXT)	45.5	$2,900	**62.8**	**$38**	**+17.3**	**77×**
Enron Emails (TXT)	**93.8**	$425	86.9	**$12**	**-6.9**	**35×**
Wiki NBA (HTML complejo)	44.8	$13,000	**68.2**	**$60**	**+23.4**	**217×**
SWDE Movie (8 webs HTML)	45.2	$5,660	**56.8**	**$46**	**+11.6**	**123×**
SWDE Univ. (5 webs HTML)	53.8	$3,800	**59.0**	**$38**	**+5.2**	**100×**
Promedio Global	**56.6**	**$5,157 (258M tok)**	**66.7**	**$39 (1.9M tok)**	**+10.1 (+18%)**	**110×**

Resultado 2: Comparación contra el Estado del Arte Supervisado y Ablación (Tablas 2 y 4)



*Nota: En SWDE ClosedIE (media Movie/Univ), *Evaporate-Code+* (76.6 Text F1) supera a *DOM-LM* (70.0) y *RoBERTa-Structural* (47.1) sin usar ningún dato de entrenamiento del dominio.

5. Análisis Crítico: Ventajas, Limitaciones y Retos Abiertos — Apartado 5

- **¿Por qué *Evaporate-Code+* supera en calidad al propio LLM directo (*Direct*)?** Al analizar los fallos de *Evaporate-Direct* en informes médicos FDA, el LLM olvida de media el **27,5% de los atributos** por documento, **alucina 9,7 atributos/valores inexistentes** por informe y renombra el mismo atributo de 4 formas distintas (*"classification"*, *"device classification"*, *"regulatory info"*). En cambio, las funciones Python sintetizadas imponen **consistencia determinista y alta precisión** (78,5% Precision frente a 58,9% Recall en Tabla 1).
- **Limitación 1 — Textos con baja regularidad sintáctica (Caso Enron):** En *Enron Emails*, *Evaporate-Direct* (93.8 F1) supera a *Evaporate-Code+* (86.9 F1). Cuando el atributo requiere comprensión semántica libre en vez de patrones estructurales/HTML/regex repetitivos, las funciones Python pierden cobertura.
- **Limitación 2 — Escalabilidad en esquemas ultra-anchos y dependencia del oráculo:** Si el esquema supera $m > 2.500$ atributos, sintetizar código por atributo deja de ser rentable frente a *Direct*. Además, el Algoritmo 1 usa al propio LLM en *D_eval* como pseudo-ground-truth: si el LLM falla sistemáticamente en un atributo complejo (tasa de error e alta en Proposición 1), filtra todas las funciones candidatas y omite la columna.

> **Hipótesis del Grupo (*EVAPORATE-Hybrid*) y Protocolo de Medición:** En Tabla 1, *Code+* logra alta Precisión (**78,5%**) pero bajo Recall (**58,9%**) por abstenciones en casos borde/texto libre.
> **Hipótesis H_1:** Enrutar al LLM con esquema fijo (*Closed-Schema Fallback*) solo las celdas donde F' se abstiene y $e > \tau$ (o confianza $P(\hat{y}|\Psi) < \theta$) elevará el Recall y F1 manteniendo un ahorro >30×.
> **Cómo medirlo:** Evaluar *ΔRecall*, *ΔPair F1* y *ΔF1/Millón-Tokens* variando el presupuesto $\beta \in \{0\%, 5\%, 10\%, 20\%\}$ frente a *Code+* (β=0%) y un fallback aleatorio. En nuestra simulación alcanza **75,0 Pair F1 (+8,3 pts)** con **38,2× de ahorro**.